

Minion 沙箱机制原理

概述

Minion 项目提供了多层次的 Python 代码执行沙箱机制，用于在 AI Agent 场景下安全地执行由 LLM 生成的代码。项目实现了从轻量级到重量级的多种隔离方案，满足不同场景的安全需求。

沙箱架构层次



1. Docker 容器沙箱 (PythonEnv)

1.1 工作原理

Docker 容器沙箱是最高安全级别的方案，通过将代码执行完全隔离在 Docker 容器中实现安全隔离。



1.2 核心组件

PythonEnv 类 (minion/main/python_env.py)

```
class PythonEnv(IntercodeEnv):
    """
    Python 代码执行环境（基于 Docker 容器）
    通过 rpyc 远程过程调用与容器内的 Python 服务器通信
    """

    def __init__(self, image_name: str, **kwargs):
        # 配置端口映射
        kwargs["ports"] = {f"{HOST_PORT}/tcp": HOST_PORT}
        super(PythonEnv, self).__init__(image_name, **kwargs)
        # 建立 rpyc 连接
        self.conn = rpyc.connect("localhost", HOST_PORT)

    def exec_action(self, action: str) -> None:
        # 通过 rpyc 在容器内执行代码
        self.observation = self.conn.root.execute(action)
```

容器内服务器 (docker/utils/python_server.py)

```
class MyService(rpyc.Service):
    def __init__(self):
        self.globals = InheritedGlobals(ORIGINAL_GLOBAL)

    def exposed_execute(self, command):
        # 提取 ID 和命令
        full_id, command = self.extract_id_and_command(command)
        namespace = self.get_namespace(full_id)

        # 在隔离的命名空间中执行
        with lock:
            exec(command, namespace)

        return {"output": output, "error": error}
```

1.3 安全特性

特性	说明
进程隔离	代码在独立容器中执行，与主进程完全隔离
文件系统隔离	容器有独立的文件系统，无法访问主机文件
网络隔离	可配置容器网络策略

特性	说明
资源限制	Docker 可配置 CPU、内存限制
命名空间隔离	支持多用户会话隔离

1.4 使用方法

```
# 构建 Docker 镜像
docker build -t intercode-python -f docker/python.Dockerfile .

# 使用
from minion.main.python_env import PythonEnv
env = PythonEnv(image_name="intercode-python")
observation, reward, done, info = env.step("print('Hello')")
```

2. RPyC 远程沙箱 (RpycPythonEnv)

2.1 工作原理

RPyC 沙箱通过远程过程调用在独立进程中执行代码，无需 Docker 即可实现进程隔离。



2.2 核心组件

RpycPythonEnv 类 (minion/main/rpyc_python_env.py)

```
class RpycPythonEnv(IntercodeEnv):
    def __init__(self, **kwargs):
        # 连接到 rpyc 服务器
        self.conn = rpyc.connect(
            kwargs.get("host", "localhost"),
            kwargs.get("port", HOST_PORT)
        )

    def step(self, action: str) -> None:
        full_id, code = extract_id_and_command(action)
        # 在远程服务器执行代码
        self.observation = self.conn.root.execute(f"<id>{full_id}</id>{code}")
```

2.3 命名空间隔离机制

服务器端支持多会话隔离：

```
def get_namespace(self, full_id):
    """基于 ID 获取隔离的命名空间"""
    id_parts = full_id.split("/")
    with lock:
        current_namespace = vars
        for part in id_parts:
            if part not in current_namespace:
                current_namespace[part] = InheritedGlobals()
            current_namespace = current_namespace[part]
    return current_namespace
```

2.4 使用方法

```
# 启动 rpyc 服务器
python docker/utils/python_server.py --port 3007

# 使用
from minion.main.rpyc_python_env import RpycPythonEnv
env = RpycPythonEnv(port=3007)
```

3. AST 解释器沙箱 (LocalPythonExecutor)

3.1 工作原理

AST 解释器沙箱是项目中最精细的安全控制方案。它不使用 Python 的 `exec()` 函数，而是：

1. 将代码解析为抽象语法树 (AST)
2. 遍历 AST 节点，逐个评估执行
3. 在每个节点执行前进行安全检查

```
Python 代码 → AST 解析 → 节点遍历 → 安全检查 → 受控执行
                        |
                        ▼
                    白名单验证
                    资源限制检查
                    危险操作拦截
```

3.2 核心安全机制

3.2.1 模块导入白名单

```
# 基础允许的模块
BASE_BUILTIN_MODULES = [
    "collections", "datetime", "itertools", "math",
    "queue", "random", "re", "stat", "statistics",
    "time", "unicodedata",
]

# 危险模块黑名单
DANGEROUS_MODULES = [
    "builtins", "io", "multiprocessing", "os",
    "pathlib", "pty", "shutil", "socket",
```

```

    "subprocess", "sys",
]

```

3.2.2 函数调用白名单

```

BASE_PYTHON_TOOLS = {
    "print": custom_print,
    "isinstance": isinstance,
    "range": range,
    "float": float,
    "int": int,
    "bool": bool,
    "str": str,
    # ... 其他安全的内置函数
}

# 危险函数黑名单
DANGEROUS_FUNCTIONS = [
    "builtins.compile",
    "builtins.eval",
    "builtins.exec",
    "builtins.globals",
    "builtins.locals",
    "builtins.__import__",
    "os.popen",
    "os.system",
    "posix.system",
]

```

3.2.3 资源限制

```

MAX_OPERATIONS = 10000000    # 最大操作次数
MAX_WHILE_ITERATIONS = 1000000 # while 循环最大迭代次数
DEFAULT_MAX_LEN_OUTPUT = 50000 # 最大输出长度

```

3.2.4 属性访问控制

```

def nodunder_getattr(obj, name, default=None):
    """禁止访问双下划线属性"""
    if name.startswith("__") and name.endswith("__"):
        raise InterpreterError(f"Forbidden access to dunder attribute: {name}")
    return getattr(obj, name, default)

```

3.3 AST 节点评估

核心的 `evaluate_ast` 函数处理各种 AST 节点：

```

@safer_eval
def evaluate_ast(expression, state, static_tools, custom_tools,
    authorized_imports):
    """评估 AST 节点"""

    # 操作计数检查（防止无限循环）

```

```

if state["_operations_count"]["counter"] >= MAX_OPERATIONS:
    raise InterpreterError("Reached max operations")
state["_operations_count"]["counter"] += 1

# 根据节点类型分发处理
if isinstance(expression, ast.Assign):
    return evaluate_assign(expression, ...)
elif isinstance(expression, ast.Call):
    return evaluate_call(expression, ...)
elif isinstance(expression, ast.Import):
    return evaluate_import(expression, ...)
# ... 其他节点类型

```

3.4 导入授权检查

```

def check_import_authorized(import_to_check: str, authorized_imports: list[str])
-> bool:
    """检查模块导入是否被授权"""
    current_node = build_import_tree(authorized_imports)
    for part in import_to_check.split("."):
        if "*" in current_node:
            return True # 通配符授权
        if part not in current_node:
            return False
        current_node = current_node[part]
    return True

```

3.5 使用方法

```

from minion.main.local_python_executor import LocalPythonExecutor

executor = LocalPythonExecutor(
    additional_authorized_imports=["numpy", "pandas"],
    max_print_outputs_length=10000,
)

# 注入可用工具
executor.send_tools({"my_tool": my_function})

# 执行代码
output, logs, is_final = executor("result = 1 + 2")

```

4. 本地执行环境 (LocalPythonEnv)

4.1 工作原理

轻量级的本地执行环境，使用 Python 的 `exec()` 函数在受控的命名空间中执行代码。

```

class LocalPythonEnv(IntercodeEnv):
    def __init__(self, **kwargs):
        self.local_env = {}
        self.reset_local_env()

```

```
def execute_code(self, code: str) -> Dict[str, str]:
    """在本地环境执行代码"""
    stdout_capture = StringIO()
    stderr_capture = StringIO()

    with redirect_stdout(stdout_capture), redirect_stderr(stderr_capture):
        exec(code, self.local_env)

    return {
        "output": stdout_capture.getvalue(),
        "error": stderr_capture.getvalue()
    }
```

4.2 特性

- **变量隔离**: 代码在独立的命名空间中执行
- **输出捕获**: 自动捕获 stdout 和 stderr
- **智能输出**: 自动显示变量赋值结果
- **表达式包装**: 自动用 `print()` 包装需要输出的表达式

4.3 安全警告

 `LocalPythonEnv` 在同一进程中执行代码，安全性较低，仅适用于可信代码。

5. 异步沙箱 (AsyncPythonExecutor)

5.1 工作原理

异步版本的 AST 解释器沙箱，支持异步工具调用和协程执行。

```
class AsyncPythonExecutor:
    """异步 Python 代码执行器，支持异步工具"""

    async def __call__(self, code_action: str) -> tuple[Any, str, bool]:
        output, is_final_answer = await evaluate_async_python_code(
            code_action,
            static_tools=self.static_tools,
            custom_tools=self.custom_tools,
            state=self.state,
            authorized_imports=self.authorized_imports,
        )
        return output, str(self.state["_print_outputs"]), is_final_answer
```

5.2 异步工具支持

```
async def evaluate_async_call(call, state, static_tools, custom_tools,
authorized_imports):
    """异步版本的函数调用评估"""

    if isinstance(func, AsyncBaseTool):
        # 异步工具直接返回协程，由 await 节点处理
        return func(*args, **kwargs)
    elif asyncio.iscoroutinefunction(func):
        # 协程函数同样返回协程对象
        return func(*args, **kwargs)
    else:
        # 同步函数直接调用
        return func(*args, **kwargs)
```

5.3 自定义 asyncio 模块

为了在现有事件循环中正确处理 `asyncio.run()` 调用：

```
def custom_run(coroutine):
    """自定义 asyncio.run(), 在现有事件循环中工作"""
    if real_asyncio.iscoroutine(coroutine):
        return coroutine # 返回协程，由 AST 评估器 await
    elif callable(coroutine):
        result = coroutine()
        if real_asyncio.iscoroutine(result):
            return result
        return result
    else:
        return coroutine
```

6. 安全机制对比

安全特性	Docker 沙箱	RPyC 沙箱	AST 解释器	LocalPythonEnv
进程隔离	✅ 容器隔离	✅ 进程隔离	❌ 同进程	❌ 同进程
文件系统隔离	✅	⚠️ 部分	❌	❌
网络隔离	✅ 可配置	❌	❌	❌
模块导入控制	❌	❌	✅ 白名单	❌
函数调用控制	❌	❌	✅ 白名单	❌
资源限制	✅ Docker 配置	⚠️ 需额外配置	✅ 内置	❌
变量隔离	✅	✅	✅	✅
设置复杂度	高	中	低	低
性能开销	高	中	低	低

7. 最佳实践

7.1 生产环境

推荐使用 **Docker 容器沙箱** 或 **AST 解释器沙箱**：

```
# 方案1: 最高安全性 - Docker 沙箱
from minion.main.python_env import PythonEnv
env = PythonEnv(image_name="intercode-python")

# 方案2: 良好安全性 + 灵活性 - AST 解释器
from minion.main.local_python_executor import LocalPythonExecutor
executor = LocalPythonExecutor(
    additional_authorized_imports=["numpy"],
    max_print_outputs_length=50000,
)
```

7.2 开发/测试环境

可使用 **RPyC 沙箱** 或 **LocalPythonEnv**：

```
# RPyC 沙箱
from minion.main.rpyc_python_env import RpycPythonEnv
env = RpycPythonEnv(port=3007)

# LocalPythonEnv (仅用于可信代码)
from minion.main.local_python_env import LocalPythonEnv
env = LocalPythonEnv(verbose=True)
```

7.3 异步场景

使用 **AsyncPythonExecutor**：

```
from minion.main.async_python_executor import AsyncPythonExecutor

executor = AsyncPythonExecutor(
    additional_authorized_imports=["asyncio", "aiohttp"],
)
executor.send_tools({"async_fetch": my_async_tool})

# 异步执行
result, logs, is_final = await executor("data = await async_fetch('url')")
```

8. 扩展和定制

8.1 添加自定义授权模块

```
executor = LocalPythonExecutor(
    additional_authorized_imports=[
        "numpy",
        "pandas",
        "sklearn.*", # 通配符支持
    ]
)
```

```
]
)
```

8.2 添加自定义工具

```
def my_custom_tool(arg1, arg2):
    """自定义工具函数"""
    return arg1 + arg2

executor = LocalPythonExecutor(additional_authorized_imports=[])
executor.send_tools({"my_tool": my_custom_tool})
```

8.3 修改资源限制

```
# 在代码中修改常量
from minion.main import local_python_executor
local_python_executor.MAX_OPERATIONS = 5000000
local_python_executor.MAX_WHILE_ITERATIONS = 500000
```

9. 总结

Minion 项目提供了完整的 Python 代码执行沙箱解决方案：

1. **Docker 容器沙箱**: 最高安全性，完全隔离，适合生产环境
2. **RPyC 远程沙箱**: 进程级隔离，无需 Docker，适合中等安全需求
3. **AST 解释器沙箱**: 精细控制，低开销，适合需要细粒度安全控制的场景
4. **LocalPythonEnv**: 轻量级，适合开发测试或可信代码执行

根据实际安全需求和性能要求，选择合适的沙箱级别，可以在安全性和易用性之间取得平衡。